

Taller de programación 2025

Unidad 2: Programación orientada a objetos

En el inicio, era el cero: vectores y matrices en Java

Hasta ahora, en Pascal, solamente habíamos trabajado con vectores. *Los **vectores** son arrays de una dimensión*, donde típicamente usamos una variable índice *i* para recorrer sus elementos. En Pascal, normalmente, un vector de N elementos tenía un índice que iba desde 1 hasta N. Pero ahora, *en Java, los vectores siempre empiezan en 0*. Por eso, un vector de N elementos irá desde el 0 hasta el N-1.

Por ejemplo, este vector de enteros que contiene 5 elementos

```
int [] vector = new int[5];
for (int i = 0; i < 5; i++) {
    vector[i] = i*2;
}
```

será así:

Contenido	0	2	4	6	8
Índice/Posición	0	1	2	3	4

Esto implica, entre otras cosas, que el tratamiento de la dimensión lógica será un poco distinto. Ahora puede inicializarse tanto en -1 como en 0, pero aparentemente lo más común (y lo más cómodo) es inicializarla en 0. Simplemente hay que acostumbrarse a que cuando uno recorre un vector, no debe recorrer hasta la dimensión lógica misma, sino hasta la posición anterior.

Ahora pasemos a las matrices.

Si los vectores son arrays de una dimensión, *las **matrices** son arrays de dos dimensiones*. Por eso, en vez de usar un sólo índice, tenemos dos (típicamente se usa *i* para las filas y *j* para las columnas).

Supongamos que queremos representar un plano simplificado para jugar a la batalla naval de 5 por 3. Eso puede entenderse como una matriz de 5 filas y 3 columnas, donde en cada posición hay un boolean (false, si no hay nada; true, si hay algún barco).

En código, quedaría así:

```
int fil = 5, col = 3;
boolean [][] matriz = new boolean [fil][col];
// Supongamos que la inicializo en false y la cargo
correctamente con algunos valores en true
```

Y visualmente podemos representarla así:

filas	columnas	0	1	2
0		false	false	false
1		false	false	false
2		false	true	true
3		false	false	false
4		true	false	false

Ahora bien, en la vida real, no usamos índices que empiezan en 0. Normalmente, todos empezamos a contar en 1. Entonces, si queremos hacer algo tipo “Si hay barco en (1, 2) [la casilla marcada con naranja], hundir el barco”, tenemos que traducir esos números a los índices de Java (es decir, restarles 1).

```
if (matriz[0][1] == true) {
    // Hundir el barco
}
```

Cambiando de tema. Supongamos que queremos recorrer una matriz completa de 3 x 4 (por ejemplo, para inicializarla en cero). Entonces, normalmente hacemos algo como esto:

```
int fil = 3; // Dimensión física de las filas
int col = 4; // Dimensión física de las columnas
int [][] matriz = new int[fil][col]; // Creamos la matriz
for (int i = 0; i < fil; i++) { // Recorremos las filas
    for (int j = 0; j < col; j++) { // Rec. las columnas
        matriz[i][j] = 0;
    }
}
```

¿Y si queremos **buscar** un elemento, cómo hacemos? Sabemos que con un *for* no, porque no necesariamente vamos a recorrer toda la matriz. Vamos a recorrerla hasta que encontremos el elemento deseado, o hasta que se termine. Así que será con un *while*. Pero como estamos un array de dos dimensiones, será con dos *whiles*.

Podríamos usar una variable auxiliar boolean, pero para mi le quita la elegancia. Así que recomiendo hacer algo así:

```
System.out.print("Ingrese un número entero: ");
valor = Lector.leerInt(); // Esto es para leer un valor desde teclado
i = 0; j = 0;

while ((i < fil) && (matriz[i][j] != valor)){
    while ((j < col) && (matriz[i][j] != valor))
        j++;
    // Si recorrió toda la fila y no lo encontró, j quedó = a col.
    // Y si eso sucede, hay que seguir buscando en la próxima fila.
    if (j == col) {
        i++;
        j = 0;
    }
}
// Si lo encontró, entonces i debe haber quedado menor a fil. De lo
// contrario, cortó porque se le terminó la matriz.
if (i != fil)
    System.out.println("Se encuentra en (" + i + ", " + j + ")");
else
    System.out.println("No se encontró el elemento");
```

Ahora bien, también hay un truquito para movernos por la matrix de otra manera. La cosa es así: si además de tener en cuenta la cantidad de filas y columnas, tenemos en cuenta la dimensión física de la matriz, podemos usar un solo índice.

La dimensión física de una matriz es el resultado de multiplicar filas por columnas.

Ahora, ¿cómo hacemos con un sólo índice? Miremos una matriz de 3 x 4, donde a cada elemento le asignamos su “posición lógica”:

fil	col	0	1	2	3
0		0	1	2	3
1		4	5	6	7
2		8	9	10	11

La dimensión física es 12, porque $3 \times 4 = 12$. Y, como podemos ver, los elementos van del 0 al 11. Hasta acá todo bien.

Ahora supongamos que queremos localizar el 6, es decir, el sexto elemento. Si observamos la tabla, está en (1, 2). Pero, ¿existe alguna operación matemática que desde el 6 nos permita llegar a ese par ordenado? Sí: si hacemos $6 \div 4$, obtenemos el 1. Y si hacemos $6 \bmod 4$, obtenemos el 2.

Es decir, para acceder al elemento n , tenemos que hacer $n \div \text{col}$ para acceder a la fila, y $n \bmod \text{col}$, para acceder a la columna.

Por eso, otra manera de recorrer la matriz es haciendo lo siguiente:

```
int fil = 3, col = 4, dimF = fil * col;
int [][] matriz = new int[fil][col];

for (int i = 0; i < dimF; i++) {
    matriz[i/col][i%col] = i;
}
```

Quizás ahora parezca más complicado e innecesario, pero hay casos donde resulta de mucha ayuda, como cuando tenemos una matriz con dimensión lógica, o cuando queremos buscar un elemento en la matriz.

Por otro lado, si queremos que, por ejemplo, cada columna tenga su propia dimensión lógica, tendremos que hacer, aparte, un vector de dimensiones lógicas.

Esto es básicamente lo que se pide en el ejercicio 4 de la práctica 2. Por eso resulta conveniente resolverlo con varias dimensiones lógicas (una por cada día), que lleven la cuenta de cuántos inscriptos hay cada día.

Acá hay una muestra de cómo podría resolverse.

```
package tema2;

import PaqueteLectura.Lector;
import PaqueteLectura.GeneradorAleatorio;

public class p2ej04 {
    public static void main(String[] args) {
        // 5 días, 8 turnos cada día
        int fil = 5, col = 8, limDNI = 50000000, limEdad = 100,
limCantTurnos = fil*col;
        int i, j, totalTurnos = 0;
        Persona [][] turnos = new Persona[fil][col];
        int [] dimLogCol = new int [fil];
        Persona p;

        System.out.println("Se van reservando turnos a medida que llegan
las personas.\n"
            + "Lo único que tiene que ingresar para cada persona es su
nombre y el día (del 1 al 5).\n"
            + "Para finalizar, ingrese el nombre ZZZ. \n");

        for (i = 0; i < fil; i++) // Se inicializan todas las dimensiones
lógicas en 0
            dimLogCol[i] = 0;

        System.out.print("El nombre: "); p = new
Persona(Lector.leerString(), GeneradorAleatorio.generarInt(limDNI),
GeneradorAleatorio.generarInt(limEdad));

        while ((totalTurnos < limCantTurnos) &&
(!p.getNombre().equals("ZZZ"))) {
            System.out.print("El día: "); i = Lector.leerInt();
            i--;
            if (dimLogCol[i] == col) {
```

```

        System.out.println("No hay más turnos disponibles para ese
día \n");
    } else {
        totalTurnos++;
        turnos[i][dimLogCol[i]] = p;
        dimLogCol[i]++;
    }
    System.out.print("El nombre: "); p = new
Persona(Lector.leerString(), GeneradorAleatorio.generarInt(limDNI),
GeneradorAleatorio.generarInt(limEdad));
}
System.out.println();

if (totalTurnos > 0) {
    System.out.println("Los turnos quedaron así: \n");
    for (i = 0; i < fil; i++) {
        System.out.println("Para el día " + (i+1) + " hay " +
(dimLogCol[i]) + " inscriptos");
        for (j = 0; j < dimLogCol[i]; j++) {
            System.out.println("El turno " + (j+1) + " es para " +
turnos[i][j].getNombre());
        }
        System.out.println();
    }
} else {
    System.out.println("No se reservó ningún turno \n");
}
}
}

```

El verdadero Java: objetos, clases, herencia y polimorfismo

Si la clave para modularizar en la programación estructurada era con funciones y procedimientos, acá va a ser con clases.

Una **clase** es como una plantilla o molde a partir de la cual creamos objetos. Por eso decimos que un **objeto** es una instancia de una clase.

Por ejemplo: si Perro es una clase que tiene nombre y raza, el labrador llamado toby (que es un objeto, una cosa concreta) será una instancia de esa clase.

Por otro lado, otra cuestión fundamental es saber delimitar bien qué hace y qué no hace cada clase. Es importante que cada clase se ocupe de lo suyo, ni más ni menos. Y si queremos que trabaje en conjunto, puede comunicarse con otras clases enviando “mensajes” (o sea, para entendernos, *parámetros*).

Pero además de comunicarse, las clases pueden formar parte de una “familia”. Es decir, una clase puede **heredar** atributos de otra clase, si ordenamos la jerarquía desde lo general a lo particular.

Por ejemplo, si queremos representar a futbolistas y basquetbolistas y de ambos tenemos DNI, edad y altura, pero de los futbolistas además tenemos club, y de los basquetbolistas tenemos entrenador, lo típico es hacer una clase más general a ambas que contenga los atributos comunes, y dos subclases que extiendan esa superclase.

Esto es clave para lograr una máxima fundamental: DRY: *Don't repeat yourself*.

Lo anoto de vuelta para que no se te olvide: DRY: *Don't repeat yourself*.

Por otro lado, está el **tema de la abstracción**. Pongamos por caso la clase Persona, que tiene DNI, edad y altura. Tranquilamente podríamos instanciar a Persona para hacer un objeto que, por ejemplo, se llame jorge, tenga dni 32.345.543 y altura 1 metro 70. Si es así, Persona no es una clase abstracta, ya que puede instanciarse.

Pero supongamos que estamos en el ejercicio de los futbolistas y basquetbolistas. Si eso es todo lo que vamos a tener en nuestro mundo (futbolistas y basquetbolistas), entonces nunca tendremos un objeto de tipo Persona, y, por lo tanto, conviene que Persona sea una clase abstracta. Esto ayuda a restringir las posibilidades para que uno no pueda hacer lo que no debe hacer. Es decir, para que no podamos siquiera cometer errores.

¿Por qué abstracta? ¿Para qué quiero eso? Porque sirve para sentar la base de subclases que sí pueden instanciarse (como Futbolista), pero ella misma no sirve para crear un objeto, no puede ser instanciada.

Ok, ¿y esto del **polimorfismo**? ¿De qué diccionario lo sacaron? La cosa es que cuando uno tiene una superclase abstracta con métodos abstractos (métodos que sólo están declarados en

la superclase, pero que sus subclases *deben* implementar a su manera), vamos a tener distintas subclases con métodos que se llaman igual, pero que hacen distintas cosas; y cuando trabajamos al nivel de generalidad de la superclase, quizás podemos llamar a ese método, pero no podemos saber en tiempo de compilación qué método en particular estamos llamando: lo sabremos recién en tiempo de ejecución, cuando ese método sea ejecutado desde alguna subclase en particular.

Si esto no se entiende nada, es porque falta el clásico ejemplo de Figura.

Supongamos que tenemos una clase Figura, tipo así:

```
public abstract class Figura {
    private String colorRelleno;
    private String colorLinea;

    public Figura(String unCR, String unCL){
        setColorRelleno(unCR);
        setColorLinea(unCL);
    }

    // Getters y setters y alguna que otra cosa más

    public abstract double calcularArea();
    public abstract double calcularPerimetro();

    @Override
    public String toString(){
        String aux = "Área: " + this.calcularArea() +
            " | Perímetro: " + this.calcularPerimetro() +
            " | CR: " + getColorRelleno() +
            " | CL: " + getColorLinea();
        return aux;
    }
}
```

Como podemos ver, Figura tiene dos métodos abstractos, pero no los implementa ella misma, ya que una figura en sí es amorfa: no tiene ni área ni perímetro. Pero las clases que sí sean alguna figura en particular (como Cuadrado o Triángulo), sí pueden responder a eso; y, de hecho, *deben* implementar esos métodos, ya que su superclase las obliga.

Por ello, la clase Cuadrado, que se *desprende* de Figura, que la *extiende*, será una cosa así:


```

public class Cuadrado extends Figura {

    private double lado;

    public Cuadrado(double unLado, String unColorR, String unColorL){
        super(unColorR,unColorL);
        setLado(unLado);
    }

    // Getter y setter de Lado

    public double calcularArea(){
        return (getLado()* getLado());
    }

    public double calcularPerimetro(){
        return (getLado()*4);
    }

    public String toString(){
        String aux = super.toString() +
                    " | Lado: " + getLado();
        return aux;
    }
}

```

Así, la clase Cuadrado tiene todo lo que está en Figura, más lo que tiene ella misma.

Ahora, ¿dónde aparece el polimorfismo?

Supongamos que tenemos un programa principal que tiene un vector de Figuras. En cada posición del vector, podremos guardar una Figura cualquiera (por ejemplo un Cuadrado, un Círculo, etc.), y podemos recorrerlo llamando a métodos que sean comunes a todos ellos (es decir, que estén implementados o declarados en Figura). Pero que estén declarados en Figura no significa que sean todos iguales: el Cuadrado calcula su área de una manera, el Círculo de otra, el Triángulo de otra, etc. Entonces cuando se ejecute el programa, cada vez que se llame `calcularArea()`, será un código distinto.

Veamos todo esto en un ejemplo concreto: un programa con un array de Figuras que calcula la suma total de sus áreas:

```

package tema4;

public class ejemploFiguras {

    public static void main(String[] args) {

        int cantFig = 3;
        Figura [] figuras = new Figura [cantFig];

        figuras[0] = new Cuadrado(10, "Violeta", "Rosa");
        figuras[1] = new Rectangulo(20, 10, "Azul", "Celeste");
        figuras[2] = new Cuadrado(30, "Rojo", "Naranja");

        double areaTotal = 0;

        for (int i = 0; i < cantFig; i++) {
            areaTotal += figuras[i].calcularArea();
        }

        System.out.println("El área total de las figuras es " + areaTotal
+ " unidades cuadradas.");

    }

}

```

Observación: pensá bien (y preguntale a la cátedra) cuándo conviene hacer un vector polimórfico y cuándo no.

Ahora volvamos un poco atrás, hasta la clase Cuadrado, para revisar un tema que nos había quedado pendiente.

Si miramos detenidamente el constructor y el `toString()`, vamos a ver una palabra clave muy particular: el *super*.

¿Y qué es esto del **super**? Lo usamos en dos situaciones:

1. Cuando en el constructor de una subclase tenemos atributos que son de la superclase, y esa superclase ya tiene un constructor hecho para esos atributos, conviene llamarlo así: `super(atributo1, atributo2, atributoN)`.
2. Cuando queremos llamar a un método que existe tanto en nuestra clase como en la superclase, y queremos desambiguar y asegurarnos que queremos el de la superclase.

El segundo punto es importante. Quizás queremos llamar a un método que está en la superclase y entonces decimos `super.getColor()` o algo así. Y va a funcionar. Pero conceptualmente está mal: porque Java sabe ir a buscarlo. Si no lo encuentra en la clase en la que está (por ejemplo, supongamos que está en la clase Cuadrado), entonces irá a buscarla a la superior (en nuestro caso, a Figura, y lo va a encontrar).

Ahora bien: si tenemos un caso como el `toString()`, por ejemplo, que existe tanto en Cuadrado como en Figura, y estamos en Cuadrado pero queremos usar el de Figura, ahí sí, necesariamente, tenemos que poner `super.toString()` para indicar que queremos usar el que está en la clase superior.

Así que fijate bien antes de usarlo: solamente hay que usarlo cuando es necesario, no cuando quieras darle una mano a Java. Vos no te preocupes. Java se maneja.

A modo de cierre, aquí van tres enseñanzas que me quedaron de las últimas clases, sobre todo para pensar cómo encarar los ejercicios y la escritura de los programas.

1) Todo lo que pueda ser privado, tiene que ser privado. En particular, si los setters pueden ser privados, mejor.

O sea, obviamente depende del caso. Si, por ejemplo (como en el ejercicio 1) te piden hacer un triángulo con setters y getters para todo, ahí sí, obviamente esos son públicos.

Pero si tenés que hacer una habitación (como en el ejercicio 4), ahí hay que pensarlo un poco. Uno no debería poder setear a gusto si la habitación está ocupada o no, ya que se podrían ocasionar errores (como cargar un cliente y poner que la habitación está libre). Uno debería hacer un método para agregar un cliente, y que ese método se encargue de todo: de chequear si la habitación está libre; de setear el cliente; y de setear que ahora quedó ocupada. (Algo similar habría que hacer si uno quiere sacar un cliente de una habitación).

En este ejemplo de la habitación, conviene que los setters sean privados, porque la idea es que la clase habitación interactúa con el exterior de la forma más simple posible, y siempre hay que pensar que el usuario puede equivocarse, y que no tiene ganas de complicarse la vida (si quisiera hacer eso se pondría a estudiar Java y no nos necesitaría).

Por otro lado, también pueden haber casos donde la clase tenga datos privados que el usuario desde afuera no debería poder ni configurar ni consultar. La dimensión lógica, por ejemplo, no debería ser siquiera posible inicializarla: convendría que por defecto siempre se inicialice en 0.

En resumen, siempre que puedas, hacé que los setters sean privados.

2) Todo lo que le corresponda hacer al objeto, que lo haga el objeto. La idea es que la interacción con el exterior sea lo más simple posible.

Esto está conectado con lo anterior. Volvamos al ejercicio 4. Una de las cosas que pide es que, en determinado momento, uno pueda aumentar el precio de todas las habitaciones.

Uno podría pensar que eso es un problema a resolver del hotel, ya que es el que tiene el vector con las habitaciones, y que entonces tiene que tomar el precio de cada una, sumarle un monto, y setear ese nuevo monto.

Un posible código (un método para el hotel) sería así:

```
public void aumentarPrecios (double monto) {
    for (int i = 0; i < dimF; i++) {
        habitaciones[i].setCosto(habitaciones[i].getCosto() + monto);
    }
}
```

Resuelve el problema, pero no es la mejor solución. Lo ideal sería que la propia habitación sepa aumentarse el precio a sí misma, y que el hotel solamente tenga que pedirle eso mandándole un monto.

Entonces tendríamos dos métodos: uno para la habitación:

```
public void aumentarPrecio(double monto) {
    this.setCosto(this.getCosto() + monto);
}
```

y otro para el hotel:

```
public void aumentarPrecios (double monto) {
    for (int i = 0; i < dimF; i++) {
        habitaciones[i].aumentarPrecio(monto);
    }
}
```

Además, otra ventaja de esta versión es que nos permite dejar el `setCosto()` de la habitación como un método privado.

- 3) No les gusta que los parámetros de los constructores se llamen igual que las variables de la clase. Prefieren que no nos veamos obligados a usar el *this* en esos casos.**

O sea, en vez de hacer esto (que es lo que NetBeans puede generar de forma automática):

```
public Autor (String nombre, String biografia, String origen) {  
    this.nombre = nombre;  
    this.biografia = biografia;  
    this.origen = origen;  
}
```

prefieren algo como esto:

```
public Autor (String unNombre, String unaBiografia, String unOrigen) {  
    setNombre(unNombre);  
    setBiografia(unaBiografia);  
    setOrigen(unOrigen);  
}
```

De todas formas, no es algo 100% obligatorio. Es más una cuestión de estilo.

Un último detalle: si queremos redondear un `double` para que tenga solo dos decimales, podemos usar esto:

```
(Math.round((NUMERO_DOUBLE) * 100.0) / 100.0)
```

No nos lo enseñó la cátedra, pero tampoco le molesta que lo usemos.